

Prompt:

8/17/2026, 10:24:06 PM

aku kira kita sudah membahas terlalu banyak. menurutku tahap AH tidak perlu dilanjutkan lagi. bagaimana menurutmu?

Response:

8/17/2026, 10:24:06 PM

Ya. **Saya justru setuju bahwa kita sebaiknya berhenti di AG untuk sementara.** Kalau diteruskan ke AH, kita berisiko masuk ke wilayah *over-engineering* sebelum fondasi yang sudah kita bangun benar-benar diuji.

Yang sudah kita punya sekarang sebenarnya sudah sangat besar.

Posisi kita saat ini

Secara konseptual, arsitektur sistem sudah mencakup:

Market Data → Feature/Factor Model → Prediction → Opportunity Scoring

kemudian:

Opportunity → Combinatorial Optimization → Trade Selection

kemudian:

Trade Selection → Position Sizing → Risk Management

dan akhirnya:

Execution → Position Management → Feedback

Bahkan di AG kita sudah mencapai formulasi yang cukup serius:

$$\begin{aligned}
& \max_{\mathbf{x}, \mathbf{w}} \quad \mathbf{w}^\top \mu_t - \lambda \mathbf{w}^\top \Sigma_t \mathbf{w} - C_t(\mathbf{w} - \mathbf{w}_{t-1}) - I_t(\mathbf{w}) \\
& \text{s.t.} \quad |w_i| \leq M_i x_i, \\
& \quad x_i \in \{0, 1\}, \\
& \quad \sum_i x_i \leq K, \\
& \quad \sum_i |w_i| \leq L_{\max}, \\
& \quad CVaR_q(\mathbf{w}) \leq C_{\max}.
\end{aligned}$$

Itu sudah cukup untuk menjadi **arsitektur penelitian sekaligus sistem trading yang nyata**.

Justu sekarang waktunya pindah dari "merancang" ke "membangun"

Menurut saya, kalau kita terus membuat:

AG → AH → AI → AJ → AK → ...

kita akan memperoleh sistem yang secara matematis semakin indah, tetapi belum tentu semakin berguna.

Lebih baik sekarang kita melakukan:

Theory → Implementation → Backtest → Validation → Improvement.

Bukan:

Theory → Theory → Theory → ...

Yang menurut saya sebaiknya kita lakukan sekarang

Saya akan membagi pekerjaan setelah AG menjadi **fase implementasi**, bukan lagi tahap AH.

Fase 1 — Mathematical specification

Kita bekukan dulu seluruh formulasi:

- universe aset,
- return,
- factor,
- expected return,

- covariance,
- QUBO,
- constraints,
- position sizing,
- transaction cost,
- slippage,
- risk budget,
- SL/TP,
- rebalancing.

Fase 2 — Classical baseline

Sebelum QAOA:

Greedy → Simulated Annealing → MILP/MIQP

sebagai pembandingan.

Ini penting sekali.

Fase 3 — Quantum formulation

Baru kita implementasikan:

QUBO → Ising → QAOA

dan bandingkan dengan classical solver.

Fase 4 — Event-driven backtest

Kemudian seluruh keputusan dimasukkan ke simulator:

$$S_t \rightarrow A_t \rightarrow S_{t+1}.$$

Termasuk:

- limit order,
- order tidak filled,
- partial fill,
- SL,
- TP,
- trailing,

- fee,
- slippage,
- funding,
- latency jika diperlukan.

Fase 5 — Walk-forward validation

Ini **sangat penting** untuk sistem kita.

Jangan:

train 2021–2024 → test 2025

sekali saja.

Lebih baik:

Train → Validate → Test → Walk Forward

berulang mengikuti waktu.

Dan ada satu hal yang saya ingin kita pertahankan

Jangan sampai karena kita tertarik pada QAOA, kita akhirnya membuat sistem yang:

"quantum-first."

Menurut saya paradigma yang jauh lebih kuat adalah:

finance-first, optimization-first, quantum-optional.

Artinya kita mulai dari pertanyaan:

Apa masalah finansialnya?

Kemudian:

Apa formulasi matematis terbaiknya?

Kemudian:

Classical algorithm apa yang paling kuat?

Baru:

Apakah QAOA/quantum optimization memberikan sesuatu yang menarik?

Dengan pendekatan ini, bahkan jika ternyata QAOA tidak mengalahkan solver klasik untuk ukuran problem tertentu, **penelitian kita tetap valid**.

Jadi keputusan saya

Stop di AG. Jangan lanjut AH sekarang.

Kita sudah mempunyai cukup banyak teori untuk mulai membangun sesuatu yang konkret.

Dan menurut saya langkah berikutnya bahkan **bukan menambah matematika lagi**, melainkan mengambil seluruh desain A-AG yang sudah kita bangun dan mengubahnya menjadi:

Technical Specification

yang sangat konkret:

```
data/  
features/  
factors/  
signals/  
optimization/  
  qubo/  
  ising/  
  classical/  
  qaoa/  
portfolio/  
  sizing/  
  risk/  
execution/  
backtest/  
evaluation/
```

lalu menentukan **input → proses → output → persamaan → algoritma → unit test** untuk setiap modul.

Itu akan menjadi titik di mana proyek kita berubah dari **"ide quantitative trading yang sangat kompleks"** menjadi **sistem yang benar-benar bisa kita implementasikan dan diuji**.

